

# 파이썬 기초 — LLM 개발에 필요한 것만

실전 API 개발을 위한 핵심 문법

오늘 배우는 모든 문법은 오후 API 실습에서 바로 사용됩니다

LLM 개발 교육과정

# 목차

## 01 학습 목표

파이썬 기본 자료형 이해 및 LLM 코드에서의 역할 파악

## 02 파이썬 기본 자료형

문자열, 정수, 실수, 불리언 및 리스트 자료형의 이해와 활용

## 03 변수 관리

LLM 설정값을 변수로 관리하고 하드코딩 방지하기

## 04 함수와 타입 힌트

반복 로직을 함수로 묶고 타입 힌트와 기본값 활용하기

## 05 딕셔너리와 API

API 요청 구조를 딕셔너리로 표현하고 중첩 접근 마스터하기

## 06 실습 및 정리

딕셔너리와 JSON의 관계 이해 및 실습 체크포인트 확인

# 학습 목표



## 기본 자료형과 변수

- 파이썬의 기본 자료형(str, int, float, bool)을 이해하고 LLM 코드에서의 역할 설명
- 변수의 역할을 이해하고 LLM 설정값을 변수로 관리



## 함수와 딕셔너리

- 타입 힌트와 기본값을 포함한 함수 작성
- 딕셔너리로 API 요청 구조를 표현하고 값을 읽고 수정



## 중첩 구조 접근

- 중첩 딕셔너리(리스트 안 딕셔너리)에서 원하는 값을 정확히 접근
- API 응답 파싱 및 데이터 추출

# 파이썬 기본 자료형 (1)

## 문자열 (str)

- 텍스트 데이터를 표현하는 자료형
- LLM 코드에서 모델명, 프롬프트, 응답 텍스트 등에 사용
- 예시: 'gpt-4o-mini', '당신은 법률 전문가입니다.'

## 정수 (int)

- 소수점이 없는 숫자를 표현하는 자료형
- max\_tokens, 반복 횟수 등에 사용
- 예시: 512, 100, 1000

## 실수 (float)

- 소수점이 있는 숫자를 표현하는 자료형
- temperature, top\_p 등 파라미터 조절에 사용
- 예시: 0.7, 0.3, 1.5

## 불리언 (bool)

- 참(True) 또는 거짓(False)을 표현하는 자료형
- GPU 연결 여부, 조건 판단 등에 사용
- type() 함수로 자료형 확인 가능

# 파이썬 기본 자료형 (2) - 리스트

## 리스트 기본 개념

- 대괄호 [] 안에 여러 값을 순서대로 담는 자료형
- API의 messages가 리스트 구조
- 인덱스는 0부터 시작

## 값 읽기와 추가

- 값 읽기: list[인덱스]
- 값 추가: list.append(값)
- 리스트 맨 끝에 새로운 항목 추가

## 길이 확인

- len(list) 함수로 전체 항목 개수 반환
- 리스트의 크기를 파악할 때 사용

## 반복 처리

- for item in list: 구문으로 항목을 하나씩 처리
- 리스트의 모든 요소를 순회하며 작업 수행

# 변수 — LLM 설정값 관리

## 변수의 역할

- 자주 바뀌는 값을 이름으로 묶어두는 것
- 모델명, 온도값, 프롬프트, 최대 토큰 등을 변수로 관리
- 실험마다 바뀌는 값을 효율적으로 관리

## 하드코딩 vs 변수 관리

- 하드코딩: 코드 여러 곳에 값을 직접 입력 → 수정 시 여러 곳 변경 필요
- 변수 관리: 변수 한 곳만 변경하면 전체에 반영
- 유지보수성과 가독성 향상

## 변수 이름 짓기 규칙

- 역할이 드러나는 이름 사용 (예: model\_name)
- 소문자 + 밑줄 snake\_case 방식 (예: max\_tokens)
- 상수는 대문자로 표기 (예: MODEL)

## 실무 활용

- 모델을 바꿀 때 변수 하나만 수정하면 전체 코드에 반영
- 코드 중복을 없애고 수정이 용이한 구조 설계
- 좋은 코드의 시작은 변수 관리부터

# 함수 — 반복 호출 로직을 하나로 묶기



## 함수 구성 요소

- 함수명: 동사로 표현
- 매개변수: 입력 정의
- 타입 힌트: 자료형 명시



- 기본값: 생략 시 사용값

## API 호출 로직 함수화

- 반환값: 실행 결과
- API 호출은 동일한 구조 반복
- 이를 함수로 묶어 중복 제거
- 수정은 한 곳만 하면 됨



- 재사용성과 유지보수성 향상

## 함수명 작성 규칙

- 함수명은 동사형(예: call\_llm)
- 수행 동작을 명확히 표기
- 소문자와 밑줄(snake\_case) 사용
- 역할이 한눈에 파악되도록 작성

# 타입 힌트와 기본값 매개변수

## 타입 힌트의 역할

- 함수가 어떤 자료형을 받고 돌려주는지 명시
- 가독성: 한눈에 파악 가능
- 협업: 다른 사람이 볼 때 실수 방지
- IDE 지원: 자동완성 및 오류 탐지 기능 향상

## 기본값 매개변수

- 자주 쓰는 값을 미리 설정
- 호출 시 생략 가능
- 예: `temperature=0.7, max_tokens=512`
- 필요할 때만 값을 변경하여 사용

## 호출 방식의 유연성

- 모든 기본값 사용: `call_llm(질문)`
- 첫 번째만 변경: `call_llm(질문, 0.0)`
- 키워드 인자로 명시: `call_llm(질문, temperature=0.0)`
- 선택적으로 원하는 매개변수만 변경 가능

## LLM 코드에서의 반환값

- 반환값 없음: 화면 출력만 하고 끝남
- 반환값 있음: 결과를 변수에 저장하고 재사용
- LLM 코드에서는 반환값이 있는 함수를 주로 사용
- API 응답을 다음 처리에 넘겨야 하기 때문

# 딕셔너리 기본 개념



## 딕셔너리란?

- 키:값 쌍으로 데이터를 저장하는 구조
- 중괄호 {}로 생성
- 예: person = {'name': '김진환', 'role': 8}
- API 요청/응답 구조와 1:1 대응

## 접근 방식

- 리스트: 순서(인덱스)로 접근
- 예: topics[0]
- 딕셔너리: 이름(키)으로 접근
- 예: person['name'] → '김진환'

## 키와 값

- 키(key): 값을 찾기 위한 이름 (문자열)
- 값(value): 키에 연결된 데이터 (모든 자료형 가능)
- 접근: 대괄호 안에 키를 넣으면 값이 나옴
- 예: dict[key]

# API 요청 구조와 딕셔너리

## API 요청 본문과 딕셔너리

- OpenAI API 요청 본문은 JSON 형식
- 파이썬에서는 딕셔너리로 표현
- 딕셔너리를 익숙하게 다루는 것이 API 개발의 절반
- 실제 API 통신에서 그대로 사용됨

## model 키

- 역할: 어떤 모델을 사용할지 지정
- 자료형: str (문자열)
- 예: 'gpt-4o-mini'
- API 요청 시 필수 파라미터

## messages 키

- 역할: 대화 내용 전체를 담음
- 자료형: list[dict] (리스트 안에 딕셔너리)
- role 값: system, user, assistant
- 멀티턴 대화 구조 표현

## temperature와 max\_tokens

- temperature: 창의성 조절 (0.0~2.0), float
- max\_tokens: 최대 출력 토큰 수, int
- 태스크에 따라 값을 조절하여 사용
- 선택적 파라미터로 기본값 설정 가능

# 딕셔너리 핵심 조작 및 중첩 접근



## 값 읽기

- dict[key]: 키가 반드시 있어야 함
- dict.get(key, 기본값): 키가 없으면 기본값 반환
  - 키 존재 확인: key in dict
- API 응답에서 텍스트 추출 시 사용



## 값 수정/추가

- 문법: dict[key] = 값
- 기존 키가 있으면 값 수정
- 없으면 새로운 키-값 쌍 추가
- 요청 파라미터 동적 변경에 활용

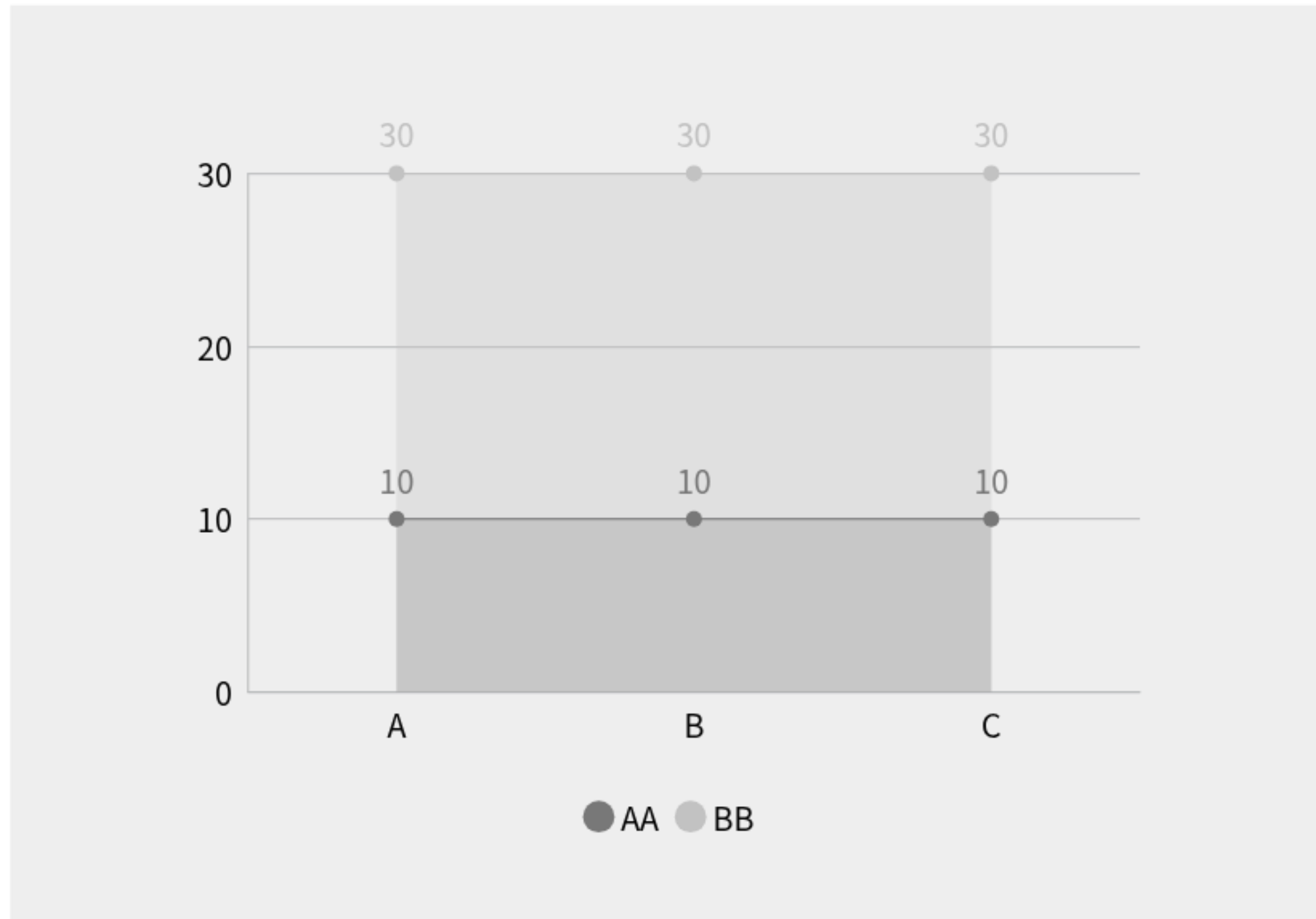


## 중첩 접근

- 리스트 안 딕셔너리 구조
  - 예:  
`request_body['messages'][0]['role']`
- 단계별 분해: 딕셔너리 → 리스트 → 딕셔너리
  - 인덱스[0]를 먼저 접근해야 함

딕셔너리 조작은 API 개발의 핵심입니다. dict[key]는 키가 반드시 있을 때 사용하고, dict.get(key, 기본값)은 키가 없을 수도 있을 때 안전하게 사용합니다. 값을 수정하거나 추가할 때는 dict[key] = 값 형식을 사용하며, 중첩 딕셔너리에서는 단계별로 접근해야 합니다. 특히 API의 messages 구조처럼 리스트 안에 딕셔너리가 있을 때는 반드시 인덱스를 먼저 지정한 후 키에 접근해야 합니다.

# 딕셔너리와 JSON의 관계 및 실습 체크포인트



## 딕셔너리와 JSON의 관계

- 파이썬 딕셔너리와 JSON은 생김새가 거의 같음
- API는 JSON 형식으로 통신, 파이썬은 딕셔너리로 처리
- True/False는 JSON에서 true/false로 표기
- None은 JSON에서 null로 표기
- json 모듈이 두 형식 간 변환을 담당

## 실습 체크포인트

- 변수 4개 선언 및 type() 확인
- 기본값 있는 함수 정의 및 호출
- request\_body['model'] 출력
- request\_body['messages'][0]['role'] 출력
- 딕셔너리 값 수정 후 재출력
- .get()으로 기본값 처리
- 리스트 반복문으로 항목 출력
- 오후 API 실습에서 실제 활용